

JavaScript

Introduction

Overview

JavaScript is a versatile, powerful programming language used for interactive web applications. It supports both client-side and server-side development, integrating seamlessly with HTML, CSS, and a rich standard library.

Dynamic Typing

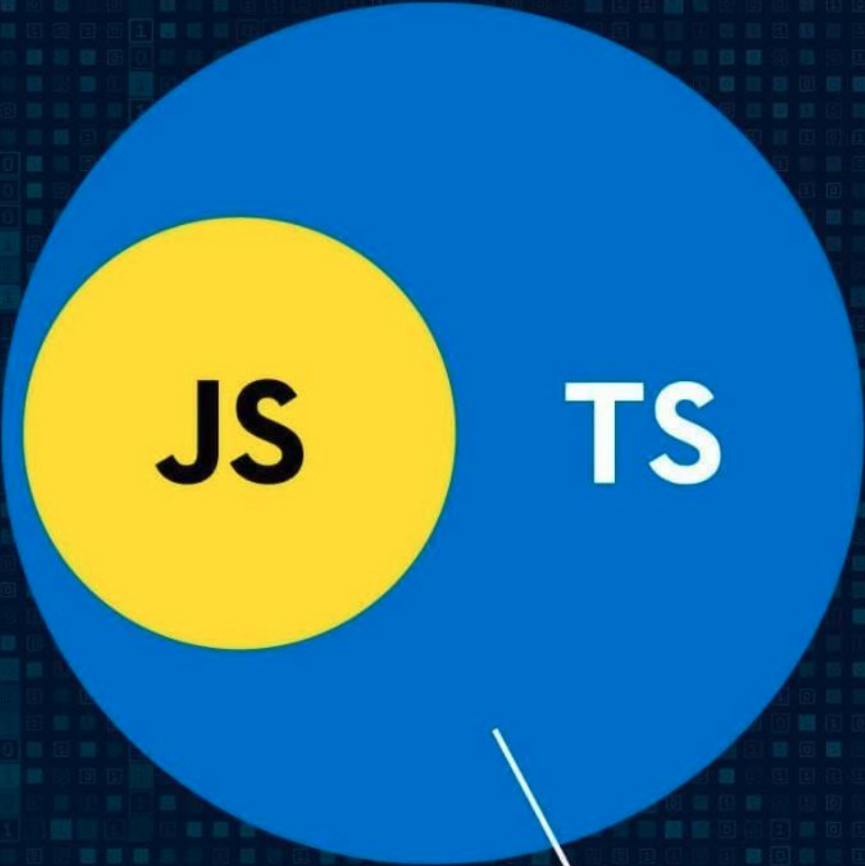
The data type of a variable is decided at run-time in JavaScript.

This is why it is called dynamically typed. You do not need to explicitly declare types before execution; the engine assigns them on the fly.

JS Engine

A JavaScript engine is the program inside the browser that reads, interprets, and executes JavaScript code.

BROWSER	JAVASCRIPT ENGINE	DEVELOPED BY
Google Chrome	V8 and Blink	Google
Microsoft Edge	V8 (via Chromium)	Google / Microsoft
Brave	V8 (via Chromium)	Brave (Chromium-based)
Opera	V8 (via Chromium)	Opera Software
Mozilla Firefox	SpiderMonkey	Mozilla
Apple Safari	JavaScriptCore (Nitro)	Apple
Samsung	V8	Samsung (Chromium-based)
IE (Old)	Chakra (retired)	Microsoft



A Venn diagram with a large blue circle and a smaller yellow circle inside it. The yellow circle is labeled 'JS' and the blue circle is labeled 'TS'. A white line points from the bottom of the blue circle to the word 'Types'.

JS

TS

Types

TypeScript

JavaScript

```
let x: number = 5
```

```
// x = "hello"
```



Type error

vs

```
let x = 5;
```

```
x = "hello;
```

Node.js

Run JavaScript Outside the Browser

Node.js is a free, open-source runtime environment that allows developers to execute JavaScript code on servers, local machines, and backend systems.

FREE & OPEN SOURCE

Supported by a massive global community and ecosystem of packages (NPM).

V8 ENGINE POWERED

Compiles JavaScript directly to native machine code for lightning-fast execution.



Data Types

0142

Number

02"abc"

String

03true

Boolean

04undefined

Undefined

05null

Null

06{}

Object

07[]

Array

08()=>{}

Function

Variable

01 / LEGACY

var

Function-scoped declaration. Historically used before ES6. Can be redeclared and updated freely within its scope.

02 / BLOCK SCOPED

let

Modern block-scoped declaration. Introduced in ES6. Can be updated with new values but cannot be redeclared.

03 / CONSTANT

const

Block-scoped constant. Introduced in ES6. Must be initialized immediately and cannot be updated or redeclared.

var

THE OLD WAY (BEFORE 2015)

Legacy Behavior

var was the only way to declare variables in JavaScript before the ES6 standard was introduced.

You can **redeclare** and **update** variables freely, which can lead to accidental overwrites.

```
var name = "Mukesh";  
  
var name = "Otwani"; // Redeclared  
  
console.log(name);
```

```
> "Otwani"
```

No errors are thrown even though the variable was redeclared.

let

THE MODERN WAY (ES6 / 2015)

Block-Scoped

let was introduced in ES6 (2015) and is now the preferred way to declare variables in most cases.

You can **update** the variable, but **can't redeclare** it in the same block.

```
let age = 30;
```

```
age = 31; // Allowed
```

```
let age = 32; // Not allowed
```

```
> SyntaxError: Identifier 'age' has  
already been declared
```

const

THE MODERN WAY (ES6 / 2015)

Immutable

const was introduced in ES6 alongside let. It is used when a variable should not change once assigned.

You can **neither update** nor **redeclare** it — its value is constant.

```
const role = "QA";
```

```
role = "Dev"; // Not allowed
```

```
> TypeError: Assignment to constant variable.
```

hoisting

COMPILATION BEHAVIOR

Hoisting is JavaScript's behaviour where declarations are moved to the top of their scope during the compilation phase.

- **var:** Hoisted and initialized as `undefined`
- **function:** Fully hoisted; callable before definition
- **let & const:** Hoisted but uninitialized (`ReferenceError`)

HOW IT WORKS IN PRACTICE

// What you write:

```
console.log(name); // undefined  
var name = "Mukesh";
```

// How JavaScript interprets it:

```
var name; // hoisted  
console.log(name); // undefined  
name = "Mukesh";
```